# Data Structures and Algorithms, Spring 2026

# Homework 0

TA E-mail: dsa_ta@csie.ntu.edu.tw

**Due: 13:00:00, Tuesday, April 21, 2026**

## Rules and Instructions

Welcome to DSA 2026 from your teaching team with 22 TAs and 2 instructors. Please start the exciting journey by carefully reading the following rules for this homework set. Many rules will also be applied to future homework sets.

- Any form of cheating, lying, or plagiarism will not be tolerated. Students can get zero scores and/or fail the class and/or be kicked out of school and/or receive other punishments for those kinds of misconducts.
- Discussions on course materials and homework solutions are encouraged. But you should write the final solutions alone and understand them fully. Books, notes, and Internet resources (including ChatGPT) can be consulted, but not copied from.
- Since everyone needs to write the final solutions alone, there is **absolutely no need to lend your homework solutions and/or source codes to your classmates at any time.** In order to maximize the level of fairness in this class, lending and borrowing homework solutions are both regarded as dishonest behaviors and will be punished according to the honesty policy.
- In Homework 0, the problem set contains 8 problems. All of them are programming problems modified from Introduction to Computer Programming course in NTU. Special thanks to Prof. Hsueh-I Lu and Shang-En Huang for granting us permission to use these problems.
- For problems in the programming part, you should write your code in C programming language, and then submit the code via the *DSA Judge*:

  https://dsa.csie.ntu.edu.tw/.

  Each day, you can submit up to 10 times for each problem. If not mentioned in the problem, the judge system will compile your code with

  ```
  gcc -std=c17 -O2 [id].c
  ```

- For debugging in the programming part, we strongly suggest you produce your own test-cases with programs and/or use tools like `gdb` or compile with flag `-fsanitize=address` to help you solve issues like illegal memory usage. For `gdb`, you are strongly encouraged to use compile flag `-g` to preserve symbols after compiling.

  Note: For students who use IDE (VScode, DevC++...) you can modify the compile command in the settings/preference section. Below are some examples:

  - `gcc -std=c17 -O2 [id].c -fsanitize=address # Works on Unix-like OS`
  - `gcc -std=c17 -O2 [id].c -g # use it with gdb`

- The score of the part that is submitted after the deadline will get some penalties according to the following rule:

$$Late\ Score = \max\left(\left(\frac{5 \times 86400 - Delay\ Time(sec.)}{5 \times 86400}\right) \times Original\ Score, 0\right)$$

  Please note that the "gold medal" of the class cannot be used on Homework 0 problems.

- The purpose of Homework 0 is to communicate our *expected background* for this class. If you cannot solve Homework 0 within a reasonable amount of time *by yourself*, it is very likely that you cannot solve those more challenging homework sets in the future. Then, you are *strongly encouraged* to not take this class (though you are always welcome to audit the class to learn at your own pace).

- Following the purpose above, the TAs will *not* hint to you how to solve the problems in Homework 0 via emails or during their TA hours (unlike what they will do for other homework sets in the future). They will only clarify any ambiguity in the problem descriptions. If you need discussions on any issues about Homework 0, please go to the Discord channel and discuss the issues with your classmates.

- If you really need email clarifications on the problem descriptions, you are encouraged to include the following tag in the subject of your email: `"[HW0-Px]"`, specifying the problem where you have questions. For example, `"[HW0-P1] Can` *id* `be negative?"`. Adding these tags allows the TAs to track the status of each email and to provide faster responses to you.

# Problem 1 - Digitized Summation Apparatus (10 pts)

**related concepts:** string manipulation, big integer arithmetic, array simulation

## Problem Description

You are designing a Digital Sum Accelerator (DSA). The DSA takes two binary strings as input and outputs their sum as a binary string. Your task is to implement the Digital Sum Accelerator (DSA).

## Input

The input consists of two lines. The first line contains a binary string representing the first number, and the second line contains a binary string representing the second number. Both binary strings will consist only of '0's and '1's and will not start with unnecessary '0's.

## Output

The output should be a single line containing the binary string that represents the sum of the two input binary strings.

## Constraints

- $1 \leq$ length of each binary string $\leq 10^5$

## Sample Testcases

**Sample Input 1**

```
100
110
```

**Sample Output 1**

```
1010
```

**Sample Input 1**

```
10
1
```

**Sample Output 1**

```
11
```

## Hints

It is recommended not to convert the binary strings directly into integers. Instead, simulate the addition process bit by bit, as you would perform it manually.

# Problem 2 - Memory Allocator (10 pts)

**related concepts:** recursion, enumeration, (dynamic programming)

## Problem Description

You are designing a memory management module for a custom operating system. The system provides a single contiguous memory segment of exactly $t$ kilobytes.

To manage this segment, you must divide it into smaller fixed-size blocks. The available block sizes are provided in an array *sizes*, where each size is specified in kilobytes . You do not have any limit on the number of blocks of each given size.

Your task is to calculate the total number of unique ways the memory segment can be perfectly partitioned into these blocks such that there is zero wasted space, that is, no internal fragmentation.

Two partitions are considered the same if they use the same count of each block size. The physical ordering of the blocks within the memory segment does not matter.

## Input

The first line contains two integers $t$ and $n$ separated by a space, where $t$ is the total size of the memory segment in kilobytes, and $n$ is the number of different block sizes available.

The second line is the array *sizes*, containing $n$ integers $s_1, ..., s_n$ separated by spaces, representing the available block sizes in kilobytes.

## Output

Output a single integer representing the total number of unique ways to partition the memory segment under the given constraints.

## Constraints

- $1 \leq t + n \leq 20$
- $1 \leq s_i \leq t$
- All block sizes $s_i$ are distinct.

## Sample Testcases

**Sample Input 1**

```
8 3
2 1 5
```

**Sample Output 1**

```
7
```

### Explanation 1

The 7 unique ways are as follows:

1. $\{1, 1, 1, 1, 1, 1, 1, 1\}$
2. $\{1, 1, 1, 1, 1, 1, 2\}$
3. $\{1, 1, 1, 1, 2, 2\}$
4. $\{1, 1, 2, 2, 2\}$
5. $\{2, 2, 2, 2\}$
6. $\{1, 1, 1, 5\}$
7. $\{1, 2, 5\}$

### Sample Input 2

```
5 1
3
```

### Sample Output 2

```
0
```

### Explanation 2

There is no way to partition a 5 KB memory segment using only 3 KB blocks without leaving any wasted space.

# Problem 3 - Territory (10 pts)

**related concepts:** simulation, 2D array, implementation details

## Problem Description

There are $K$ kingdoms on a rectangular land of size $H \times W$, divided into $H \times W$ equally sized cells. Each kingdom has a castle located in a distinct cell. The $K$ kingdoms wish to expand their territories across the land, subject to the following restrictions:

- Each territory must be a single rectangle composed of grid cells.
- Each territory must contain exactly one castle (its own).
- Territories belonging to different kingdoms must not overlap.
- The union of all territories must completely cover the entire $H \times W$ land.

Due to these complicated restrictions, the kingdoms need you to plan a valid land division. Specifically, a grid of $H$ rows and $W$ columns (representing the $H \times W$ land) is given. The grid contains exactly $K$ `'#'` characters (representing castles), and the rest are `'.'` characters (representing empty cells). You must assign an integer from 1 to $K$ to each cell such that:

- All cells that share the same integer must form a single valid rectangle.
- Each formed rectangle must contain exactly one `'#'` character.

Find any valid assignment of integers. It is guaranteed that a solution exists. **If multiple solutions exist, any one of them will be accepted.**

## Input

- The first line has 3 integers $H, W, K$ separated by spaces.
- The grid is represented by the following $H$ lines, each of which contains $W$ characters.

## Output

Output $H$ lines that contains $W$ integers separated by spaces that represents a valid assignment of integers.

## Constraints

- $1 \leq H, W \leq 300$
- $1 \leq K \leq H \times W$
- Each character in the grid is either `'#'` or `'.'`.
- There are exactly $K$ `'#'` characters in the grid.

## Sample Testcases

### Sample Input 1

```
3 3 5
#.#
.#.
#.#
```

### Sample Output 1

```
1 2 2
1 3 4
5 5 4
```

### Sample Input 2

```
3 3 1
...
.#.
...
```

### Sample Output 2

```
1 1 1
1 1 1
1 1 1
```

# Problem 4 - Palindrome Chain (10 pts)

**related concepts:** pointer, struct, linked list, list reversal

## Problem Description

A palindrome is a sequence that reads the same backward as forward. For example, 121 is a palindrome, while 1212 is not. Given the head of a linked list where each `ListNode` stores a single digit, determine if the sequence of digits forms a palindrome. Specifically, your implementation should adhere to the following requirements:

- You must implement your solution in the `isPalindrome` function.
- You are required to include the header file using `#include "palindrome_chain.h"` in your code to access the `ListNode` structure definition.
- You must maintain the original structure and values of the linked list. If you temporarily modify the list during processing, you must restore it to its original state before the function returns.

Your function should return `true` if the sequence is a palindrome, and `false` otherwise.

### Function Prototype

palindrome_chain.h

```h
#ifndef PALINDROME_H
#define PALINDROME_H

#include <stdbool.h>

struct ListNode {
    int val;
    struct ListNode *next;
};

bool isPalindrome(struct ListNode* head);

#endif
```

### Usage Example

You can traverse the linked list of nodes using the `next` pointer as follows:

```
struct ListNode *cur = head;
while (cur != NULL) {
    // Access data
    ...
    // Move to the next node
    cur = cur->next;
}
```

## Input

The first line is the length $N$, followed by $N$ integers. The input is managed by the grader. Your function will receive the `head` pointer of the linked list.

## Output

Return `true` if the linked list is a palindrome, and `false` otherwise.

## Constraints

- The length of the chain $N$ satisfies $0 \leq N \leq 10^5$.
- Each node's value is a single digit (within $\{0, 1, 2, \ldots, 9\}$).

## Sample Testcases

### Sample Input 1

```
3
1 2 1
```

### Sample Output 1

```
true
```

### Sample Input 2

```
4
1 2 1 2
```

### Sample Output 2

```
false
```

# Problem 5 - XOR Array (10 pts)

**related concepts:** bitwise operation, prefix XOR, range query

## Problem Description

Given an integer array $a_1, ..., a_n$ and several intervals $[l_1, r_1], \ldots, [l_m, r_m]$ of integer bounds. For each interval $[l_i, r_i]$, calculate

$$a_{l_i} \text{ XOR } a_{l_i+1} \text{ XOR } ... \text{ XOR } a_{r_i}.$$

The program must run within 1 second.

## Input

- The first line contains two integers $n$ and $m$, where $n$ is the number of elements in the array, and $m$ is the number of intervals.
- The second line contains $n$ integers $a_1, ..., a_n$, representing the elements in the array.
- The following $m$ lines each contains two integers $l_i$ and $r_i$, where the $i$-th line represents the $i$-th interval.

## Output

For each interval, output a line with a single integer, representing the XOR result of the elements in the interval.

## Constraints

- $1 \leq n, m \leq 10^5$
- $0 \leq a_i \leq 10^9$
- $1 \leq l_i \leq r_i \leq n$

## Sample Testcases

### Sample Input 1

```
4 4
1 3 4 8
1 2
2 3
1 4
4 4
```

### Sample Output 1

```
2
7
14
8
```

### Sample Input 2

```
6 7
1 1 4 5 1 4
1 1
1 2
1 3
1 4
1 5
1 6
3 5
```

### Sample Output 2

```
1
0
4
1
0
4
0
```

## Explanation of Sample 1

The binary representations of elements in the array are:

- $1 = 0001$
- $3 = 0011$
- $4 = 0100$
- $8 = 1000$

The XOR values for queries are:

- $[1, 2] = 1$ XOR $3 = 2(0010)$
- $[2, 3] = 3$ XOR $4 = 7(0111)$
- $[1, 4] = 1$ XOR $3$ XOR $4$ XOR $8 = 14(1110)$
- $[4, 4] = 8(1000)$

## Hints

Note that $a_i + ... + a_j = S_j - S_{i-1}$, where $S_i = a_1 + ... + a_i$. Can similar ideas be applied to XOR? In particular, what is the result of $p$ XOR $q$ XOR $p$?

# Problem 6 - Unit Distance Code (10 pts)

**related concepts:** gray code, recursion, pattern observation

## Problem Description

Generate a sequence consisting of all $2^n$ binary strings of length $n$ such that any two consecutive strings differ in exactly one bit.

For example, when $n = 2$, the sequence is:

$$00,\ 01,\ 11,\ 10$$

To make the sequence unique, construct each string recursively from left to right. When choosing the next bit based on the current prefix:

- If the prefix contains an **even** number of 1s, first append 0, then append 1.
- If the prefix contains an **odd** number of 1s, first append 1, then append 0.

For instance, when $n = 2$, the empty prefix contains zero 1s (even), so we explore the branch starting with 0 before the branch starting with 1. After moving to the prefix 1, the number of 1s becomes one (odd), so we explore the sub-branch 11 before 10.

## Input

The input contains one integer $n$.

## Output

Print $2^n$ lines of the code, in the exact order described above.

## Constraints

- $1 \leq n \leq 16$

## Sample Testcases

| Sample Input 1 | Sample Output 1 |
| --- | --- |
| 3 | 000<br>001<br>011<br>010<br>110<br>111<br>101<br>100 |

## Hints

Observe how the sequence changes between adjacent strings. What pattern can you find in the order of the bits?

# Problem 7 - Ingredients in Common (10 pts)

**related concepts:** string processing, set intersection, mapping

## Problem Description

In this problem, you are given a list of foods, each with its set of ingredients. Your task is to determine the common ingredients between two different foods.

You will first be given a number of foods, followed by their associated ingredients. Then, you will be asked a series of queries, each asking for the common ingredients between two specific foods. Your goal is to output the common ingredients in dictionary order for each query. If there are no common ingredients, output `nothing`.

## Input

- The first line contains an integer $n$, representing the number of foods.
- The next $n$ lines describe each food. Each line begins with the food's name, followed by an integer $i$ (the number of ingredients), and then the list of $i$ ingredients.
- The next line contains an integer $q$, representing the number of queries.
- The following $q$ lines each contain two food names, representing a query.

## Output

- For each query, output the common ingredients in dictionary order, separated by a space.
- If there are no common ingredients, output `nothing`.

## Constraints

- $1 \leq n \leq 100$
- $1 \leq i \leq 10$
- $1 \leq q \leq 10000$
- There are no repeated ingredients for any single food.
- The name of any ingredient will not be `nothing`.
- Food and ingredient names consist only of lowercase English letters and have a maximum length of 64 characters.

## Sample Testcases

### Sample Input 1

```
3
cake 4 egg flour sugar butter
omelet 4 egg bacon ham butter
bread 1 flour
2
cake omelet
omelet bread
```

### Sample Output 1

```
butter egg
nothing
```

# Problem 8 - Bingo (10 pts)

**related concepts:** simulation, 2D array, state tracking

## Problem Description

Write a program to play bingo. A bingo board has $m$ rows and $m$ columns. Each entry has a different number from 1 to $m \times m$. Each number will be called with some sequence. If a player has all the numbers in a row, a column, or a diagonal called ze can declare bingo and win the game. Now given the bingo boards of $n$ players, determine who wins the bingo.

## Input

The first line of the input has $n$ and $m$, where the integer $n$ represents the number of players and the integer $m$ represent the size of the bingo board. The second line has the name of the first player. Each of the next $m$ lines has the $m$ numbers of the bingo board of the first player. The next line has the name of the second player, and the next $m$ line have the numbers for the second player, and so on. The next line has $m \times m$ numbers that are called in sequence during the bingo game.

## Output

Print the number that that causes the first bingo followed by a space. Then, print the names of the winners of the game one by one, according to the order they appear in the input.

## Constraints

- $1 \leq n \leq 10$
- $1 \leq m \leq 256$
- The length of a name is positive and no more than 64.
- The name consist of only lowercase or uppercase English letters.

## Sample Testcases

### Sample Input 1

```
2 3
Alice
1 2 3
4 5 6
7 8 9
Blob
1 2 3
4 5 6
7 8 9
1 2 4 8 6 3 9 5 7
```

### Sample Output 1

```
3 Alice Blob
```